llnl_c1/lulesh · lulesh.cc · bare_llm_v4 · claude-sonnet-5 · FASTER

−10 / +133 lines of 1967 · 65 parallel region(s) added · 0 of 0 candidate(s) kept · RESTRUCTURING (40 code line(s) added, 10 removed) · restructuring level 0

BEFORE

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114     for (Index_t i = 0 ; i < numElem ; ++i){
115        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
116     }
117  }
118
     …
308
309     Real_t fx_local[8] ;
310     Real_t fy_local[8] ;
311     Real_t fz_local[8] ;
312
313    for( Index_t k=0 ; k<numElem ; ++k )
314    {
315      const Index_t* const elemToNode = domain.nodelist(k);
316      Real_t B[3][8] ;
317      Real_t x_local[8] ;
     …
327                              x_local, y_local, z_local );
328
329      SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
330                                   fx_local, fy_local, fz_local ) ;
331
332     for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
333        Index_t gnode = elemToNode[lnode];
334        domain.fx(gnode) += fx_local[lnode];
335        domain.fy(gnode) += fy_local[lnode];
336        domain.fz(gnode) += fz_local[lnode];
337     }
338
339   }
340
     …
495    gamma[3][4] = Real_t( 1.);
496    gamma[3][5] = Real_t(-1.);
497    gamma[3][6] = Real_t( 1.);
498    gamma[3][7] = Real_t(-1.);
499
500    for(Index_t i2=0;i2<numElem;++i2){
501       Real_t hgfx[8], hgfy[8], hgfz[8] ;
502
503       Real_t coefficient;
504
     …
607
608       CalcElemFBHourglassForce(xd1,yd1,zd1,
609                       hourgam,
610                       coefficient, hgfx, hgfy, hgfz);
611
612       domain.fx(n0si2) += hgfx[0];
613       domain.fy(n0si2) += hgfy[0];
614       domain.fz(n0si2) += hgfz[0];
615
616       domain.fx(n1si2) += hgfx[1];
617       domain.fy(n1si2) += hgfy[1];
618       domain.fz(n1si2) += hgfz[1];
619
620       domain.fx(n2si2) += hgfx[2];
621       domain.fy(n2si2) += hgfy[2];
622       domain.fz(n2si2) += hgfz[2];
623
624       domain.fx(n3si2) += hgfx[3];
625       domain.fy(n3si2) += hgfy[3];
626       domain.fz(n3si2) += hgfz[3];
627
```

AFTER

```
109  void InitStressTermsForElems(Domain &domain,
110                               Real_t *sigxx, Real_t *sigyy, Real_t *sigzz,
111                               Index_t numElem)
112  {
113
114     #pragma omp parallel for shared(domain, sigxx, sigyy, sigzz) firstprivate(n
115     for (Index_t i = 0 ; i < numElem ; ++i){
116        sigxx[i] = sigyy[i] = sigzz[i] =  - domain.p(i) - domain.q(i) ;
117     }
118  }
119
     …
309
310     Real_t fx_local[8] ;
311     Real_t fy_local[8] ;
312     Real_t fz_local[8] ;
313
314    #pragma omp parallel for private(fx_local, fy_local, fz_local) \
315            shared(domain, sigxx, sigyy, sigzz, determ) firstprivate(numElem)
316    for( Index_t k=0 ; k<numElem ; ++k )
317    {
318      const Index_t* const elemToNode = domain.nodelist(k);
319      Real_t B[3][8] ;
320      Real_t x_local[8] ;
     …
330                              x_local, y_local, z_local );
331
332     SumElemStressesToNodeForces( B, sigxx[k], sigyy[k], sigzz[k],
333                                  fx_local, fy_local, fz_local ) ;
334
335     /* adjacent elements share nodes, so the accumulation below is a
336        genuine race across elements: make each add atomic rather than
337        serializing the (expensive) math above it. */
338     for( Index_t lnode=0 ; lnode<8 ; ++lnode ) {
339        Index_t gnode = elemToNode[lnode];
340        #pragma omp atomic update
341        domain.fx(gnode) += fx_local[lnode];
342        #pragma omp atomic update
343        domain.fy(gnode) += fy_local[lnode];
344        #pragma omp atomic update
345        domain.fz(gnode) += fz_local[lnode];
346     }
347
348   }
349
     …
504    gamma[3][4] = Real_t( 1.);
505    gamma[3][5] = Real_t(-1.);
506    gamma[3][6] = Real_t( 1.);
507    gamma[3][7] = Real_t(-1.);
508
509    #pragma omp parallel for shared(domain, determ, x8n, y8n, z8n, dvdx, dvdy,
510            firstprivate(numElem, hourg)
511    for(Index_t i2=0;i2<numElem;++i2){
512       Real_t hgfx[8], hgfy[8], hgfz[8] ;
513
514       Real_t coefficient;
515
     …
618
619       CalcElemFBHourglassForce(xd1,yd1,zd1,
620                       hourgam,
621                       coefficient, hgfx, hgfy, hgfz);
622
623       /* n{0..7}si2 are shared mesh nodes reused by neighboring elements:
624          accumulate atomically instead of serializing this iteration. */
625       #pragma omp atomic update
626       domain.fx(n0si2) += hgfx[0];
627       #pragma omp atomic update
628       domain.fy(n0si2) += hgfy[0];
629       #pragma omp atomic update
630       domain.fz(n0si2) += hgfz[0];
631
632       #pragma omp atomic update
633       domain.fx(n1si2) += hgfx[1];
634       #pragma omp atomic update
635       domain.fy(n1si2) += hgfy[1];
636       #pragma omp atomic update
637       domain.fz(n1si2) += hgfz[1];
638
639       #pragma omp atomic update
640       domain.fx(n2si2) += hgfx[2];
641       #pragma omp atomic update
642       domain.fy(n2si2) += hgfy[2];
643       #pragma omp atomic update
644       domain.fz(n2si2) += hgfz[2];
645
646       #pragma omp atomic update
647       domain.fx(n3si2) += hgfx[3];
648       #pragma omp atomic update
649       domain.fy(n3si2) += hgfy[3];
650       #pragma omp atomic update
651       domain.fz(n3si2) += hgfz[3];
652
653       #pragma omp atomic update
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is